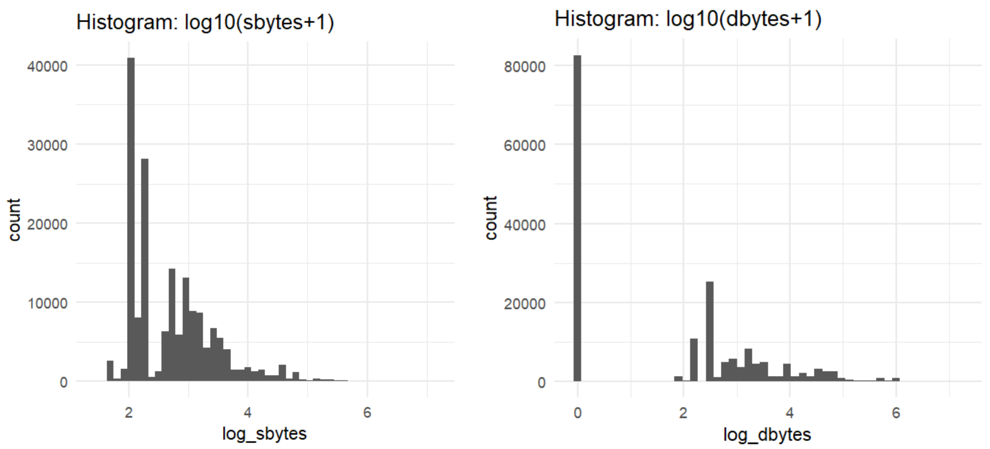
6.3 Amlan Data Analysis

The data analysis involved for this part included histograms, boxplots, scatterplots, density plots and few categories-based charts. Statistical methods such as correlation, a simple regression model, a t-test and a simple regression model was also applied for further assessment of relationships.

Distribution of sbytes and dbytes:

The distribution of the cleaned byte variables were analysed with using log-scaled histograms as it provides a clear view of the spread of its values and also reduces influence of extreme observations.

```
ggplot(df, aes(log_sbytes)) +
  geom_histogram(bins = 50) + theme_minimal() + labs(title = "Histogram: log10(sbytes+1)"
ggplot(df, aes(log_dbytes)) +
  geom_histogram(bins = 50) + theme_minimal() + labs(title = "Histogram: log10(dbytes+1)"
```

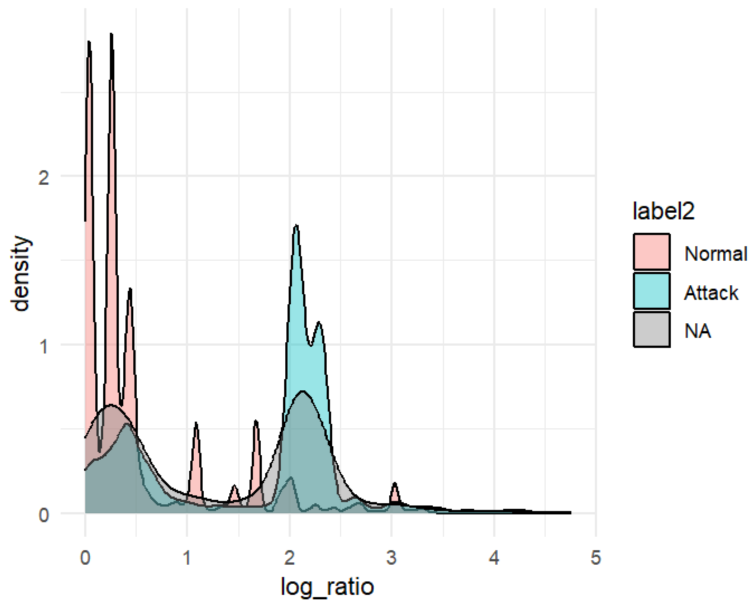


The histograms above shows that both of the sbytes and dbytes are highly skewed with most of their values concentrated near the lower part. This further indicates that a large portion of the traffic consists o small bye transfers and the involvement of the flow of high-colume data is on the smaller side.

Source–Destination Data Imbalance Across Labels:

The sd_ratio variable is used here to represent how much data is sent compared to the received. A density plot was generated to compare the difference between Normal and Attack Traffic. This will help identify if attacks involve unusual byte imbalances.

```
ggplot(df, aes(log_ratio, fill = label2)) +
  geom_density(alpha = 0.4) +
  theme_minimal()
```

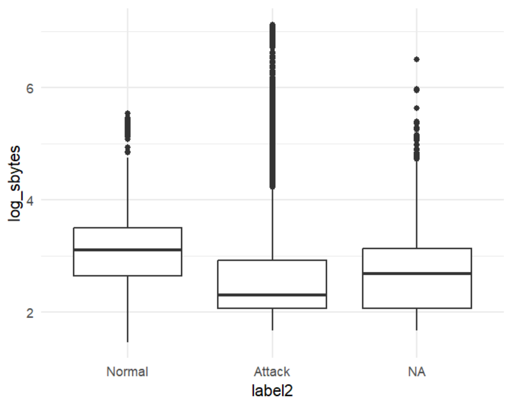


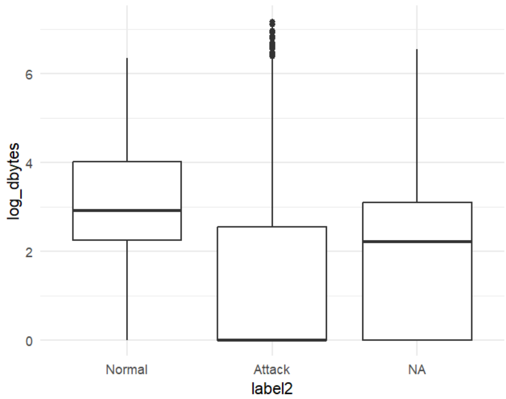
The density plot here, suggests that there is a clear difference in the imbalance ratio between Normal and Attack. This clearly supports that the assumption that imbalanced traffic is often linked to some extent of suspicious activities.

<u>Comparison Between Normal and Attack Traffic (Boxplots):</u>

Boxplots were used here to compare the log_scaled and destination bytes for Normal vs Attack traffic as well to highlight whether attacks typically generate higher or lower byte sizes.

```
ggplot(df, aes(label2, log_sbytes)) + geom_boxplot() + theme_minimal()
ggplot(df, aes(label2, log_dbytes)) + geom_boxplot() + theme_minimal()
```



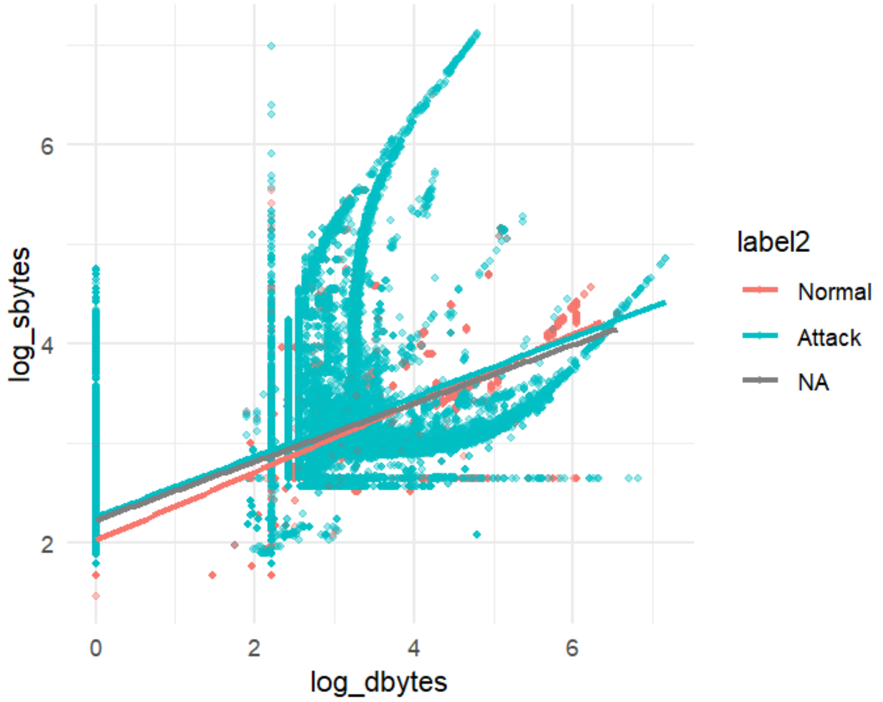


The boxplots here, gives an indication that Attack traffic generally involves higher spread and higher median values for both log_sbytes and log_sbytes compared to the Normal traffic. This suggests that the attacks do often involve some larger byte transfers or sometimes, more variable data movements.

Relationship Between sbytes and dbytes:

A scatter plot was used to observe the relationship between source and the destination bytes. The Colour-coding by traffic label allows more visual comparison of patterns for Attack and Normal flows.

```
ggplot(df, aes(log_dbytes, log_sbytes, color = label2)) +
  geom_point(alpha = 0.4, size = 1) +
  geom_smooth(method = "lm", se = FALSE) +
  theme_minimal()
```

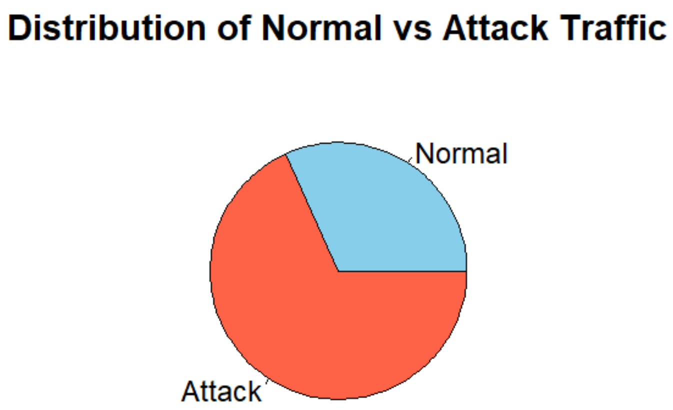


The scatterplot shows a positive relationship between destination and source, which concludes that the flows higher incoming bytes generally have higher outgoing bytes. The attack points appears to have more scattering compared to Normal, suggesting that they have less predictability.

Distribution of Normal vs Attack Traffic (Pie Chart):

A pie chart was also generated to visualize the portion of Normal and Attack traffic to provide even more clearer overview of the balance of entire dataset.

```
label_counts <- table(df$label2)
pie(label_counts,
    main = "Distribution of Normal vs Attack Traffic",
    col = c("skyblue", "tomato"))
```

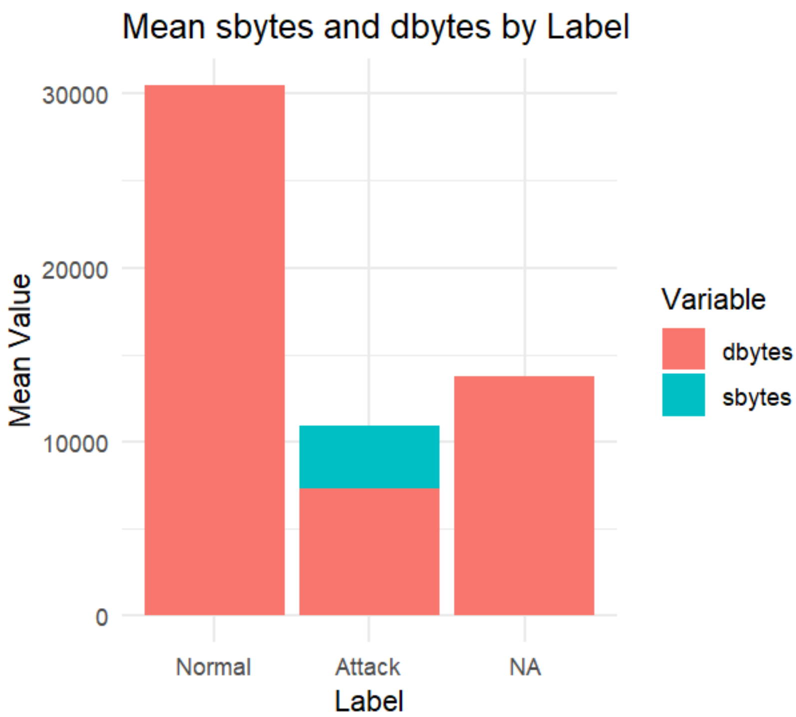


The pie chart shows that the dataset contains a higher proportion of Attack traffic compared to Normal traffic. The imbalance in the pie chart highlights the need for a careful evaluation of the classification model, as it might have learn biased patterns.

<u>Mean Byte Comparison Using Bar Chart:</u>

A bar chart was used to make a comparison between the average sbytes and dbytes for each label categories as well. This allows direct comparison of how byte usage differs between Normal and Attack.

```
mean_bytes <- df %>%
  group_by(label2) %>%
  summarise(
    mean_sbytes = mean(sbytes, na.rm = TRUE),
    mean_dbytes = mean(dbytes, na.rm = TRUE)
  )

ggplot(mean_bytes, aes(x = label2)) +
  geom_col(aes(y = mean_sbytes, fill = "sbytes"), position = position_dodge(width = 0.9)) +
  geom_col(aes(y = mean_dbytes, fill = "dbytes"), position = position_dodge(width = 0.9)) +
  labs(title = "Mean sbytes and dbytes by Label",
       x = "Label",
       y = "Mean Value",
       fill = "Variable") +
  theme_minimal()
```

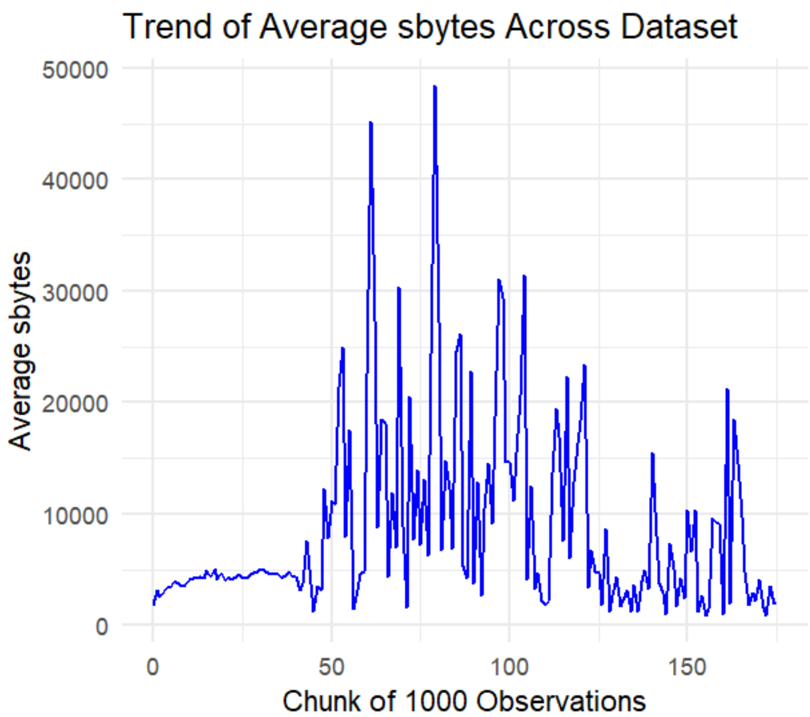


The bar chart shows that there’s a clear difference between the average sbytes and dbytes across the label categories. Noraml traffic shows the highest mean byte values, while the Attack shows lower mean values with some noticeable activity in both the directions. The additional “NA” group reflects entries with incomplete label information.

Trend of Average sbytes (Line Chart):

A line chart was generated to visualize how average sbytes changes across the whole dataset when grouped in chunks, this helps us to identify any large variation or change in byte volume throughout the different parts of the data.

```
df$index <- 1:nrow(df)

line_data <- df %>%
  mutate(chunk = index %/% 1000) %>%      # group every 1000 rows
  group_by(chunk) %>%
  summarise(avg_sbytes = mean(sbytes, na.rm = TRUE))

ggplot(line_data, aes(x = chunk, y = avg_sbytes)) +
  geom_line(color = "blue") +
  labs(title = "Trend of Average sbytes Across Dataset",
       x = "Chunk of 1000 Observations",
       y = "Average sbytes") +
  theme_minimal()
```



We can see a lot of noticeable fluctuations throughout the Line chart here. The several pointy spikes indicates that there is a period of unusually high data transfers, which may conclude that theirs is a possibility of attack activity. The overall pattern of the Line chart suggests that byte usage is not uniform and varies significantly across the dataset.

Correlation Analysis:

Correlation between log-scaled source and destination bytes was calculated to examine the strength of their linear relationship.

```
> cor(df$log_sbytes, df$log_dbytes)
[1] 0.7076541
```

The approximate correlation value of 0.71 indicates a that there's a strong positive relation between log-scaled source and destination bytes, suggesting that flows with higher outgoing data also have higher incoming too. Although the relationship is not the optimal but still it shows a consistent pattern across the dataset.

Regression Analysis:

A linear regression model was also executed to examine the strength of the relationship between log_sbytes and log_dbytes.

```
> #simple regression
> #-----------------------------------------------------------
> lm_fit <- lm(log_sbytes ~ log_dbytes, data = df)
> summary(lm_fit)

Call:
lm(formula = log_sbytes ~ log_dbytes, data = df)

Residuals:
    Min      1Q  Median      3Q     Max 
-1.6078 -0.2065 -0.1345  0.0686  4.0903 

Coefficients:
             Estimate Std. Error t value Pr(>|t|)    
(Intercept) 2.2346244  0.0016836  1327.3   <2e-16 ***
log_dbytes  0.2954636  0.0007045   419.4   <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 0.5032 on 175339 degrees of freedom
Multiple R-squared:  0.5008,	Adjusted R-squared:  0.5008 
F-statistic: 1.759e+05 on 1 and 175339 DF,  p-value: < 2.2e-16
```

The regression results helps to show that log_dbytes is a notable predictor of log_sbytes with a strong conclusive coefficient. The value of 0.5(approx.) of the R-squared indicates that around half of the contrast in log_sbytes can be explained by log_dbytes alone, although some other factors may also contribute to the remaining variation.

<u>T-Test on Byte Ratio:</u>

At last, at-test was performed to compare and analyse the mean imbalance ratio between Normal and Attack traffic.

```
> #-----------------------------------------------------------
> #t-test
> #-----------------------------------------------------------
> t.test(log_ratio ~ label2, data = df)

	Welch Two Sample t-test

data:  log_ratio by label2
t = -265.9, df = 136375, p-value < 2.2e-16
alternative hypothesis: true difference in means between group Normal and group Attack is not equal to 0
95 percent confidence interval:
 -1.084148 -1.068283
sample estimates:
mean in group Normal mean in group Attack 
           0.5589463            1.6351616 

> 
```

The t-test shows a conclusive and statistically important difference in the mean log_ratio between Normal and Attack. Attack flows have a higher imbalance ratio, suggesting that their incoming and outgoing patterns are less uniform, leading into the idea that theirs is a possibility of malicious behaviour.

<u>Additional Features (Machine Learning Method):</u>

- Decision Tree Classification

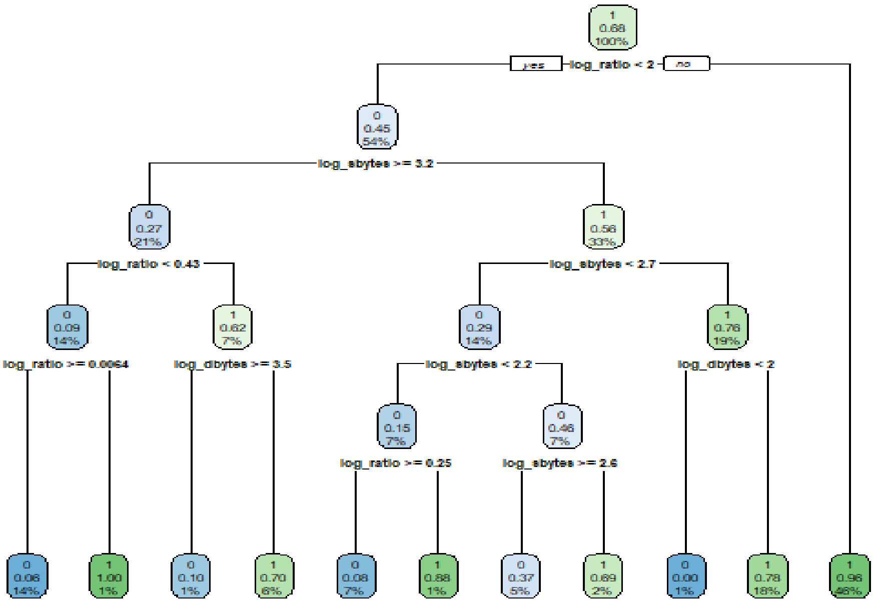
A decision tree model was applied using the rpart package that helps to classify traffic based on the log-transformed byte features. This model creates a series of rules that can separate Normal and Attack traffic by the usage of threshold values in the predictors. This classification offers a clearer structure showing which variables contributes more to the classification.

```r
#decision tree
#------------------------------------------------------------
tree_model <- rpart(label_bin ~ log_sbytes + log_dbytes + log_ratio,
                    data = train,
                    method = "class")

summary(tree_model)

#------------------------------------------------------------
#decision tree plot
#------------------------------------------------------------
plot(tree_model)
text(tree_model, use.n = TRUE)
rpart.plot(tree_model)

#------------------------------------------------------------
#decision tree prediction
#------------------------------------------------------------
tree_pred <- predict(tree_model, newdata = test, type = "class")

#------------------------------------------------------------
#decision tree confusion matrix
#------------------------------------------------------------
table(Predicted = tree_pred, Actual = test$label_bin)
```

```
> summary(tree_model)
Call:
rpart(formula = label_bin ~ log_sbytes + log_dbytes + log_ratio,
    data = train, method = "class")
  n=138036 (2236 observations deleted due to missingness)

          CP nsplit rel error    xerror        xstd
1 0.18011138      0 1.0000000 1.0000000 0.003935145
2 0.15326742      1 0.8198886 0.8198886 0.003710267
3 0.05493806      3 0.5133538 0.5822480 0.003283053
4 0.02463916      4 0.4584157 0.4414593 0.002936402
5 0.01757018      5 0.4337766 0.3982043 0.002811119
6 0.01725196      6 0.4162064 0.3882941 0.002780935
7 0.01194833      7 0.3989544 0.3706330 0.002725668
8 0.01000000     10 0.3631094 0.3441982 0.002639186

Variable importance
 log_ratio log_dbytes log_sbytes
        36         34         30

Node number 1: 138036 observations,    complexity param=0.1801114
  predicted class=1  expected loss=0.3187212  P(node) =1
    class counts: 43995 94041
   probabilities: 0.319 0.681
  left son=2 (74692 obs) right son=3 (63344 obs)
```

```
Node number 23: 25289 observations
  predicted class=1  expected loss=0.218079  P(node) =0.1832058
    class counts:  5515 19774
   probabilities: 0.218 0.782

Node number 40: 9243 observations
  predicted class=0  expected loss=0.07941145  P(node) =0.06696079
    class counts:  8509   734
   probabilities: 0.921 0.079

Node number 41: 834 observations
  predicted class=1  expected loss=0.1235012  P(node) =0.006041902
    class counts:   103   731
   probabilities: 0.124 0.876

Node number 42: 6742 observations
  predicted class=0  expected loss=0.368585  P(node) =0.04884233
    class counts:  4257  2485
   probabilities: 0.631 0.369

Node number 43: 2449 observations
  predicted class=1  expected loss=0.3062474  P(node) =0.01774175
    class counts:   750  1699
   probabilities: 0.306 0.694
```

The summary output shows how the tree splits the data at different nodes and which class each node predicts. It also reports the importance of each variable in the model and the class distribution.



The generated plot shows how the decision tree model separates Normal and Attack traffic using the log_transformed byte features.

```
> table(Predicted = tree_pred, Actual = test$label_bin)
         Actual
Predicted     0     1
        0  7937  1083
        1  2926 22536
>
```

This confusion matrix shows how many Normal and Attack flows are correctly and incorrectly classified by the decision tree model.

- Naive Bayes Model

Naive Bayes is based on learned class probabilities, which helps to conclude that whether the log-transformed byte features could be used to distinguish between Normal and Attack traffic. Including this feature adds a probabilistic view to the analysis. Naive Bayes is based on learned class probabilities, which helps to conclude that

```
> nb_model

Naive Bayes Classifier for Discrete Predictors

Call:
naiveBayes.default(x = X, y = Y, laplace = laplace)

A-priori probabilities:
Y
   Normal    Attack
0.3187212 0.6812788

Conditional probabilities:
        log_sbytes
Y             [,1]      [,2]
  Normal 3.035869 0.7054442
  Attack 2.586038 0.6703404

        log_dbytes
Y             [,1]     [,2]
  Normal 2.939476 1.411491
  Attack 1.080272 1.496728

        log_ratio
Y              [,1]      [,2]
  Normal 0.5599014 0.7096467
  Attack 1.6368078 0.9225983
```

These values show how the model takes use of the differences within the log_transformed byte variables to estimate the likelihood of the class.

```
> table(Predicted = nb_pred_class, Actual = test$label2)
         Actual
Predicted Normal Attack
   Normal   9316   8569
   Attack   1547  15050
```

The confusion matrix here shows how the Naïve Bayes classifier distinguishes between Normal and Attack traffic.